LASGen: Synergistic Harness–Seed Co-Synthesis for High-Coverage Library Fuzzing

Ming Chen
School of Cyber Science and Technology,
Shandong University
Qingdao, China
ambrose@mail.sdu.edu.cn

Yujie Xing
School of Cyber Science and Technology,
Shandong University
Qingdao, China
xyj0920@mail.sdu.edu.cn

Jiongyi Chen*
National University of Defense Technology
Changsha, China
chenjiongyi@nudt.edu.cn

Wenrui Diao*
School of Cyber Science and Technology,
Shandong University
Qingdao, China
diaowenrui@link.cuhk.edu.hk

Abstract

Library fuzzing is essential for identifying vulnerabilities in software libraries. However, achieving high coverage remains challenging due to the difficulty of generating effective harness and seed inputs. We present LASGen, an automated framework that integrates static analysis and large language models (LLMs) to generate high-quality fuzzing inputs for arbitrary library functions. LASGen generates coupled harnesses and initial seeds, treating them as a unified task rather than separate steps. This design ensures the seeds are highly compatible with the harness, enabling deeper path exploration. To achieve this, LASGen extracts function-level context via slicing and data-flow analysis, then embeds it into structured prompts to guide LLMs during synthesis, ensuring that the outputs are valid. LASGen also incorporates a self-repair mechanism and seed validation loop to ensure correctness and effectiveness. Experiments on 11 libraries with 127 known vulnerabilities show that LASGen achieves 77.17% vulnerability coverage and 55.61% edge coverage, outperforming state-of-the-art fuzzers while maintaining a lower false-positive rate. When applied to 17 real-world libraries, LASGen discovered 33 previously unknown vulnerabilities, 26 of which have been confirmed and patched, including eight assigned CNVD IDs.

CCS Concepts

• **Security and privacy** → Software and application security.

Keywords

Library fuzzing, Vulnerability detection, LLMs

ACM Reference Format:
Ming Chen, Yujie Xing, Jiongyi Chen, and Wenrui Diao. 2026. LASGen: Synergistic Harness–Seed Co-Synthesis for High-Coverage Library Fuzzing.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
ASIA CCS '26, Bangalore, India
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2356-8/26/06
https://doi.org/10.1145/3779208.3807490

In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26), June 1–5, 2026, Bangalore, India.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3779208.3807490

1 Introduction

Fuzzing is a widely used technique for identifying software vulnerabilities. Among software components, libraries are desirable targets due to their extensive reuse and exposure [22]. As a result, library fuzzing has gained increasing attention. Unlike program fuzzing, library fuzzing requires generating initial inputs for functions to achieve high code coverage [20]. These inputs typically consist of two key components: a harness that invokes the target function and an initial seed that serves as a mutation template during fuzzing. The quality of both components significantly influences the effectiveness of fuzzing.

The initial inputs for library fuzzing primarily consist of the harness and initial seed. The harness is a driver program that invokes the target function and passes mutated inputs. The initial seed serves as the initial input to the harness during fuzzing and acts as the template for subsequent mutations. These two components work in tandem and are critical for improving code coverage in library fuzzing. Although various input generation techniques [10, 12, 15, 34] have been developed, traditional static-analysis-based methods often exhibit limitations in terms of efficiency and correctness, particularly when handling complex function calls.

Recent studies [7, 8] highlight the potential of large language models (LLMs) in enhancing library fuzzing, particularly for generating initial inputs [20, 36, 42] due to their strong code semantic analysis and generation capabilities. Fuzz4All [36] generates harnesses by extracting resource information and using iterative feedback to optimize LLM prompts. PromptFuzz [20] incorporates coverage feedback to refine prompts and compose functions to generate harnesses. Jiang et al. [16] combine LLMs with constraint-based tools to improve seed quality.

However, existing approaches often fall short in generating high-quality inputs. LLMs may struggle to understand unfamiliar or poorly documented functions [4, 14], leading to incorrect or incomplete harnesses. Furthermore, most existing techniques treat

harness and seed generation as separate processes, failing to leverage their synergy. This disconnect limits the ability to effectively explore deep code paths and maximize fuzzing coverage [18, 20].

Our Work. In this work, we present LASGen, an automated framework for library fuzzing that achieves high code coverage by tightly coupling static program analysis with LLMs. Unlike prior work that loosely combines these techniques, LASGen fully automates the coupled generation of fuzzing harnesses and initial seeds. Rather than treating them as independent steps, we co-design them so that seeds are highly compatible with their harnesses, enabling deeper paths from the outset. LASGen treats static analysis not just as a preprocessing step but as a semantic guide that shapes the LLM's generation of both harnesses and seeds. Specifically, LASGen first performs whole-function slicing to extract precise semantic context for the target function and embeds this information into prompts that guide the LLM to generate valid harnesses with correct function invocation patterns. Then, it conducts data-flow analysis to infer path constraints, which are also encoded into prompts to help the LLM synthesize high-coverage synergistic seeds. To further ensure correctness and effectiveness, LASGen incorporates a self-repair module to fix compilation issues and a seed-evaluation component to filter out ineffective inputs. Finally, the generated harnesses and seeds are submitted to the fuzzing engine to discover vulnerabilities.

To evaluate LASGen, we extended the dataset used in AFGen [18] and constructed the *LibFuzz-Bench* benchmark, which includes 127 CVE-assigned vulnerabilities across 11 real-world libraries. LASGen achieved 77.17% vulnerability coverage and 55.61% edge coverage, outperforming state-of-the-art fuzzers including AFGen [18], Hopper [5], OGHarn [28], and PromptFuzz [20], while maintaining a low false-positive rate. The generated harnesses showed precision improvements of 2.26× and 1.76× over Hopper and PromptFuzz, respectively. LASGen also achieved the shortest average time to crash, outperforming G^2Fuzz [43] by 92.88%, Angr [29] by 53.80%, and the basic LLM prompt method by 87.01%. Furthermore, in real-world fuzzing of 17 libraries, LASGen discovered 33 previously unknown vulnerabilities, 26 of which were confirmed by developers, and eight of which were assigned CNVD (China National Vulnerability Database) IDs.

Contributions. The main contributions of this paper are:

- ***New Fuzzing Framework.*** We present LASGen, a high-coverage library fuzzing framework that *jointly* generates fuzzing harnesses and their corresponding seed inputs. By coupling static program semantics with LLM-assisted synthesis, LASGen produces high-quality, executable harness-seed pairs and supports a fully automated end-to-end fuzzing pipeline.
- ***Benchmark and Evaluation.*** We constructed a new benchmark *LibFuzz-Bench* for library fuzzing evaluation. Experimental results show that LASGen achieves the highest coverage and the lowest false-positive rate compared to existing tools.
- ***Real-world Vulnerabilities.*** We applied LASGen to 17 real-world libraries and discovered 33 previously unknown vulnerabilities, 26 of which were confirmed by developers.

Responsible Disclosure. We reported all discovered vulnerabilities to the corresponding libraries' developers. Of these, 26 were confirmed and patched, with eight assigned CNVD IDs: CNVD-2025-01421, CNVD-2025-01422, CNVD-2025-01634, CNVD-2025-04513, CNVD-2025-04539, CNVD-2025-23874, CNVD-2025-23868, and CNVD-2025-23869.

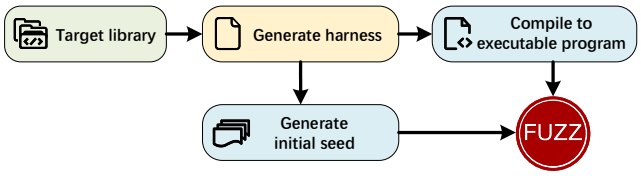


Figure 1: Workflow of typical library fuzzing.

Open Science. Our tool and the *LibFuzz-Bench* benchmark are available at `https://github.com/ambrosecm/LASGen`.

2 Background and Motivation

2.1 Library Fuzzing

Among software components, libraries have become attractive targets for attackers due to their widespread use and ease of integration [22]. Fuzzing, a widely used software testing technique, has also been applied to test libraries. Unlike general fuzzing that targets entire programs, library fuzzing involves testing multiple functions within a library. Figure 1 illustrates the typical workflow: security experts first analyze the target library to understand how its functions are invoked. They then construct a harness, generate initial seeds from it, compile the harness into an executable, and finally submit the executable along with the seeds to a fuzzing engine. To enhance the effectiveness of library fuzzing, many researchers aim to increase code coverage, which is widely recognized as a key metric for evaluating fuzzing performance [18, 20, 28]. Two main approaches are commonly adopted: (1) Monitoring the fuzzing process dynamically and providing real-time feedback, as implemented in tools such as AFL++ [9] and libFuzzer [23]; (2) Generating high-quality initial inputs that explore diverse code paths [18, 20], thereby achieving coverage of deeper code paths and more effective vulnerability detection.

2.2 Harness and Initial Seed

The inputs to library fuzzing comprise two main components: the harness and the initial seed. The harness is a driver program that delivers mutated seeds to the target function, requiring correct invocation and a reliable input interface. However, creating effective harnesses is challenging due to the large number of functions and their complex usage patterns. Manual construction requires a deep understanding of the domain and is time-consuming. To address this, researchers have developed various automated harness generation methods [5, 12, 15], which analyze function signatures and call relationships to synthesize harnesses. Nonetheless, achieving both high code coverage and ensuring the generated harnesses are correct remains a significant challenge.

Initial seeds are the starting inputs for fuzzing and serve as templates for subsequent mutations. Their quality strongly influences

the fuzzing process in terms of direction, efficiency, and vulnerability uncovering. Randomly generated seeds often lead to early input rejection [1, 33, 41], resulting in poor coverage and limited vulnerability discovery. Some approaches [10, 34] leverage symbolic execution or dynamic analysis to generate more targeted seeds. Still, these techniques face limitations, including path explosion and complex constraint solving [19, 39], thereby limiting their practical effectiveness.

2.3 LLM for Library Fuzzing

High-quality inputs ensure both correctness and coverage. Traditional static analysis-based input generation often struggles with efficiency and scalability, particularly for complex function calls. Recent research [7, 8, 20] highlights the potential of large language models (LLMs) to enhance library fuzzing. When equipped with relevant program knowledge, LLMs can leverage their semantic understanding and generation capabilities to efficiently produce initial inputs. Consequently, LLMs are increasingly used to generate fuzzing inputs [20, 36, 38, 42]. However, LLMs still fall short of autonomously producing consistently high-quality fuzzing inputs.

On the one hand, studies have shown that LLMs suffer from issues like hallucination [4, 14, 40]. Since LLMs rely on learned knowledge to predict outputs, an insufficient understanding of function behavior often leads to incorrect calls, resulting in compilation errors and false positives. Techniques such as Retrieval-Augmented Generation (RAG) [2] and prompt engineering [35] have been proposed to improve accuracy, but effectively delivering valuable knowledge and guidance to LLMs remains challenging. On the other hand, due to the vast number of library functions and their complex invocation patterns, guiding LLMs to cover more functions and more deeply into code paths remains difficult [20]. This often results in limited coverage and shallow exploration in the generated inputs.

Current Limitations. Overall, existing approaches still face substantial limitations in generating high-quality fuzzing inputs that simultaneously satisfy correctness and achieve high coverage. Specifically, although the introduction of LLMs alleviates some difficulties encountered by traditional techniques and increases the success rate of input generation, the lack of contextual knowledge and hallucination issues often lead to function misuse, resulting in false positives during fuzzing. In addition, existing work on library fuzzing typically suffers from low coverage and shallow exploration. Current fuzzing input generation approaches for libraries generally focus on the API level, which limits the number of functions covered. Moreover, they always treat seed generation as an independent task separate from the harness, overlooking their synergistic relationship. This design results in insufficient input constraints, which often yields unusable seeds and hinders normal fuzzing execution. Compounding this, without guidance from the target function's path constraints, the generated seeds are frequently rejected early during fuzzing and fail to explore deeper execution paths. As a result, these approaches are ineffective at exposing vulnerabilities hidden in deep code paths.

Our Approach. To address these challenges, we propose a practical, synergistic fuzzing input-generation approach that integrates static analysis with LLM-based generation. To mitigate hallucinations and function misuse, our method extracts precise function-call contexts and intra-procedural data-flow constraints from the library via static analysis. This accurate, semantically aware information is embedded in prompts, replacing the LLM's reliance on unreliable prior knowledge with valid guidance for function invocations. This significantly reduces invalid function calls and false positives during input synthesis. To overcome limited coverage and shallow exploration, we incorporate whole-function slicing with per-function generation prompts to ensure comprehensive coverage of each target function. Moreover, we fully leverage the synergy between the harness and the seeds. We use the data flow extracted from the function execution paths, together with the harness's contextual information, to steer LLMs toward generating seeds that satisfy more stringent path constraints. This enables the generated seeds not only to remain valid but also to traverse more complex and deeper execution paths, thereby increasing overall code coverage.

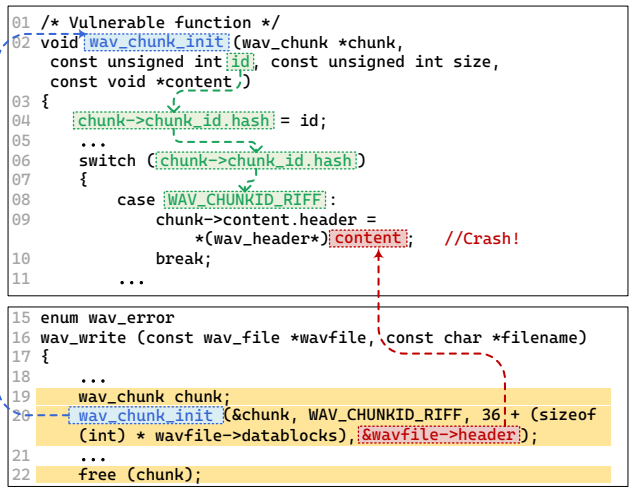


Figure 2: Motivation example.

Figure 2 presents a motivating example, illustrating how real-world libraries often contain complete and correct function invocation patterns. In the example code snippet, a vulnerability exists at `Line 9`, where the `wav_chunk_init` function is invoked with the `WAV_CHUNKID_RIFF` ID. If the `content` pointer refers to an invalid address, an invalid memory access occurs. Notably, the `wav_write` function includes a complete and correct invocation of `wav_chunk_init`, encompassing parameter definition, flag setting with `WAV_CHUNKID_RIFF`, buffer allocation, data passing, and memory release.

Such contextual information can be systematically extracted from the library to guide LLMs in generating more effective fuzzing inputs with broader function coverage. Furthermore, the execution path of `wav_chunk_init` provides a complete parameter data flow that can be used to derive constraints for seed construction, enabling deeper exploration of execution paths from the outset of fuzzing.

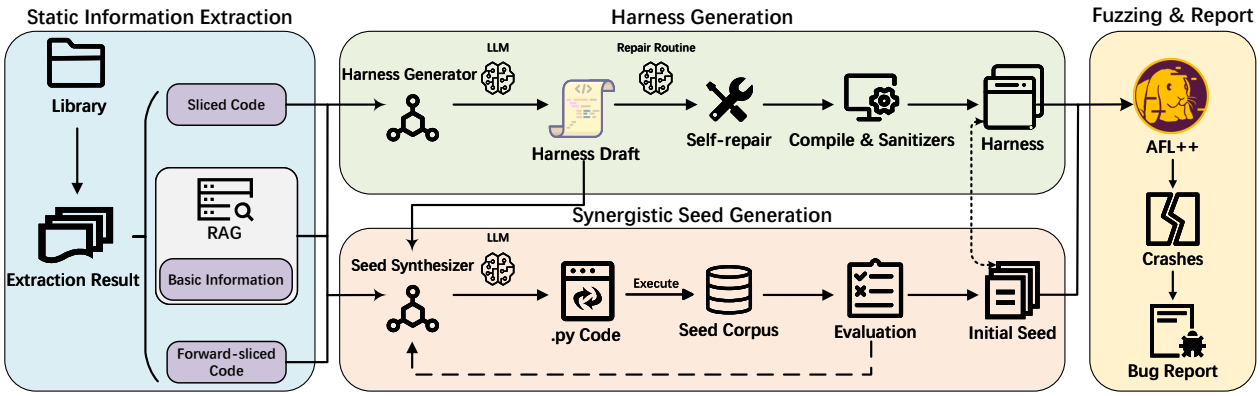


Figure 3: Overview of LASGen Fuzzing.

3 Design of LASGen

This section presents the detailed design of LASGen. The overall architecture is illustrated in Figure 3, which consists of three stages:

1. ***Static Information Extraction***: This module performs static analysis on the target library to extract function information, which supports the subsequent generation stages.
2. ***Harness Generation***: Based on the results of static analysis, this module generates harnesses aimed at covering the complete set of library functions.
3. ***Synergistic Seed Generation***: This module incorporates function dataflow guidance to produce synergistic seeds that are both executable and capable of reaching deep code paths.

The generated harnesses and seeds are ultimately submitted as the initial input corpus to the fuzzing engine. Crashes produced during fuzzing are then organized and analyzed to identify potential vulnerabilities.

3.1 Static Information Extraction

To generate fuzzing inputs, LASGen learns the invocation procedures and input compositions of relevant functions from statically extracted information. Therefore, static analysis is first performed on the target library to obtain this knowledge. The extraction process involves three main components: *library information extraction*, *target function slicing*, and *dataflow-guided forward slicing*.

(1) Library Information Extraction. The extracted information includes function metadata, function definitions, and other relevant descriptions. This data serves as the foundation for both slicing and input generation, as outlined below:

Function Metadata Table: Records essential attributes for each function, including its signature, parameter list, source file path, and slicing count. These attributes are used to indicate the source and sink functions for slicing and to record intermediate states.

Function Definition: Stores the complete source code of each function, which is used in subsequent slicing stages.

Function Call Graph: A directed graph with functions as nodes and call relationships as edges, providing the function dependencies necessary for constructing accurate execution paths during forward slicing.

Compile Commands: Includes build instructions, compiler flags, and linked libraries, which are used to compile the generated harness into executable programs.

Knowledge Vector Repository: A vectorized knowledge base from library documentation for LLM guidance.

LASGen begins by analyzing the library's source code. It traverses all source files to extract function symbols, signatures, file paths, and other related metadata, which are stored in the *Function Metadata Table*. An additional field in the table tracks the number of times each function has been sliced. Meanwhile, LASGen extracts function definitions, which are required in later stages for input generation. To enable cross-file slicing, LASGen constructs a directed function-call graph of the entire library. Each node in the graph represents a function and contains its associated metadata, while each edge denotes a call relationship between functions. The specific process is shown in Algorithm 1.

(2) Target Function Slicing. To assist LLM-based harness generation, LASGen provides sliced code capturing the invocation context of the target function. This sliced code serves as input knowledge to guide accurate and complete generation. To ensure comprehensive coverage of all functions in the library, LASGen traverses the function metadata table and performs slicing for each listed function. Due to the presence of infeasible execution paths and redundant syntactic structures in source code, direct source-level analysis is challenging. In contrast, the Intermediate Representation (IR) provides a standardized, semantically consistent structure, facilitating accurate program analysis. Therefore, slicing is performed on bitcode files.

The slicing process is illustrated using the example in Figure 2. Suppose LASGen selects `wav_chunk_init` as the target function from the metadata table. According to the function call graph, `wav_write` is identified as the slicing entry point. LASGen begins

Algorithm 1 Cross-File Function Call Graph Generation

Input: Function metadata table $parsed_funs$, Function call database $func_db_file$
Output: Directed function call graph $G(V, E)$

```
Initialize empty directed graph G ← (V, E)
for all function f ∈ parsed_funs do
    current_node ← (f.symbol, f.file)
    add current_node to G.V
    c_funs = get_called_funs(f.symbol, func_db_file)
    for all ⟨c_sym, c_file⟩ ∈ c_funs do
        found_exact ← False
        for all function f′ ∈ parsed_funs do
            if is_exist(c_sym, f′.symbol, f′.file) then
                found_exact ← True
            end if
        end for
        if found_exact then
            target_node ← (c_sym, c_file)
            if target_node ∉ G.V then
                add target_node to G.V
            end if
            add edge ⟨current_node, target_node⟩ to G.E
        end if
    end for
end for
return G
```

from this entry, locates the call site of the target function, and traces dependencies along the path to collect all the code that affects the control flow of the invocation. To retrieve the variable initializations required for the function call, LASGen performs a data-flow analysis. Starting at the entry point, it slices operations that influence the input parameters of the target function, such as definitions (e.g., `wav_chunk`), assignments (e.g., `WAV_CHUNKID_RIFF`), computations (e.g., `wavfile->datablocks`), and parameter passing (e.g., `free(chunk)`). The resulting code fragments are then post-processed to restore original symbols and semantics, and the necessary headers are added to ensure the sliced code is compilable.

The yellow-highlighted region in Figure 2 shows the final sliced code for the `wav_chunk_init` function. After each slicing operation, the `slice_count` field in the function metadata table is incremented for the corresponding function. A slicing limit is enforced to avoid redundant or excessive slicing of the same function.

(3) Dataflow-guided Forward Slicing. Beyond slicing the function call, LASGen also learns the path constraints of functions to improve branch coverage during seed generation. Therefore, it is necessary to perform forward slicing to trace the data flow originating from the function's arguments.

The forward slicer starts at the call site of the target function and focuses on tracking the downstream effects of its parameters. As shown in the green box in Figure 2, the forward slicing process for the `wav_chunk_init` function starts by identifying its input parameters. The slicer then traces the data dependencies associated with these parameters (e.g., `id`) to identify the affected code regions. The cross-file forward slicing process proceeds as follows:

① Retrieve the complete call chain of the target function from the function call graph.
② Slice all code with data dependencies along the path from the function's entry point.
③ Identify invoked functions in the sliced code and treat them as new slicing entry points.
④ Repeat until the entire call chain has been processed.

3.2 Harness Generation

Once the static analysis of the library is complete, LASGen generates harnesses. Our approach is based on the intuition that correct invocation patterns for target functions already exist within the library. By combining sliced invocation code and static information with LLM-guided prompt generation, LASGen enhances both the correctness and coverage of the generated harnesses. The overall harness generation process comprises two components: *harness draft synthesis* and *harness self-repair*.

(1) Harness Draft Synthesis. LASGen integrates sliced invocation code and extracted function metadata to generate the harness draft through carefully designed prompts.

To guide the LLM effectively, LASGen designs a dynamically updated prompt template. In the template, LASGen assigns an LLM the role of a fuzzing expert and specifies the desired output format. The prompt includes chain-of-thought (CoT) reasoning and example-based guidance to encourage step-by-step synthesis. Prior to harness generation, LASGen automatically embeds the pre-extracted sliced code into the prompt. The LLM is instructed to complete the sliced code into a valid function invocation rather than generating it from scratch, thereby reducing common invocation errors in code generation. Function metadata is also incorporated to help the LLM understand argument passing and generate appropriate input-parsing code based on parameter types. The prompt template used for generating harness drafts is illustrated in Figure 5 in Appendix A.3.

Additionally, a hybrid Retrieval-Augmented Generation (RAG) mechanism retrieves relevant documentation from the knowledge base, providing auxiliary context for the target function. Before sending the prompt to the LLM, LASGen checks its token count. If the prompt exceeds the token limit, the context is segmented and simplified to fit within the model's constraints. Upon submission, a harness containing a valid call to the target function is synthesized.

During this process, some functions may lack sufficient sliced code, affecting harness generation. To handle such edge cases, LASGen traverses the function metadata table and identifies functions with a slicing count of zero. For these functions, an alternative prompt template is used, instructing the LLM to generate harnesses based on the function definition and retrieved RAG knowledge. This strategy enables LASGen to generate harness drafts for any function in the library.

An example of the harness generated for `wav_chunk_init` is shown in Listing 1. Lines 8–17 demonstrate the construction of arguments based on data read from the input file and processed to match the function's parameter format. Line 20 shows the initialization of the `wav_chunk` variable. On line 22, `wav_chunk_init` is invoked with the `WAV_CHUNKID_RIFF` flag. Finally, line 24 deallocates memory once the variables are no longer needed.

```c
#include <stdio.h>
#include <stdlib.h>
#include "libwav.h"

int main(int argc, char *argv[])
{
    // Open the input file
    FILE *inputFile = fopen(argv[1], "rb");
    ...
    // Read the file content into a buffer
    fseek(inputFile, 0, SEEK_END);
    long fileSize = ftell(inputFile);
    fseek(inputFile, 0, SEEK_SET);
    void *fileContent = malloc(fileSize);
    ...
    fread(fileContent, 1, fileSize, inputFile);
    fclose(inputFile);

    // Initialize the wav_chunk structure
    wav_chunk chunk;
    // call wav_chunk_init function
    wav_chunk_init(&chunk, WAV_CHUNKID_RIFF, fileSize,
        fileContent);
    // Free the allocated memory
    free(fileContent);
    ...
```

Listing 1: Harness for `wav_chunk_init` invocation.

(2) Harness Self-repair. The harness generated in the previous stage must be compiled into an executable before fuzzing can begin. However, due to the inherent randomness of LLM-based generation, it is often difficult to guarantee successful compilation on the first attempt. To improve the compilation success rate, LASGen introduces a harness self-repair module.

This module designs a fully automated repair routine: First, LASGen retrieves the appropriate compilation command and attempts to compile the harness. If the compilation succeeds, the process proceeds to the fuzzing stage. Otherwise, the harness enters a self-repair loop. In this loop, the compilation error messages, along with the repair generation prompt, are passed to a specialized code-repair LLM. This LLM is instructed with prompts that include error-specific repair strategies, enabling it to autonomously select how to correct compilation errors and regenerate the harness. Representative examples of these strategies are shown in Table 1. Once the repaired harness is produced, it is recompiled. This loop repeats until the harness compiles successfully or a predefined retry limit is reached. If the code remains unfixable after the allowed number of attempts, the harness is discarded, and a new generation phase is initiated.

3.3 Synergistic Seed Generation

After generating the harness for a target function, LASGen produces high-quality initial seeds. Guided by the data flow along the target function's execution path and the harness's contextual information, LASGen generates seeds that satisfy the harness's input path constraints. These seeds are compatible with the generated harness and work collaboratively with it. They improve code coverage for the target function during fuzzing and enable the fuzzer to explore deeper execution paths more effectively. The seed generation process is based on the intuition that the forward-sliced code of a function reflects the data flow and control paths of its parameters. By embedding this sliced code, the LLM can better derive the input

Table 1: Strategies for repairing compilation errors.

Error type	Repair strategy
Standard Library Error	Incorrect header order. Arrange headers: standard libraries first, target libraries last.
Duplicate Inclusion	Remove redundant header inclusions.
Undefined Reference	Change library header extensions from `.c` to `.h` or remove problematic `.c` files.
Undefined Function	Check for missing header references.
Syntax Error	Fix code based on compiler error messages.
API Redefinition	Avoid redefining APIs implemented in headers.

Table 2: Input type classification.

Classification	Parameter Type
Normal	Int, Float, Double, Byte, Char, String, Bool
Complex	JSON, XML, CSV, JPEG, PNG, PDF, TIFF, WAV, BIN, YAML, DOCX, MP3, MP4, SVG

constraints associated with various execution branches, thereby generating seeds that exercise a broader range of the function's logic. The seed generation process consists of three main steps: *input type classification*, *seed corpus synthesis*, and *seed validation and corpus finalization*.

(1) Input Type Classification. Since different harnesses require different parameter types, the contents written to seed files must be adapted accordingly. To address this, LASGen classifies the harnesses based on input types before seed generation to satisfy specific input requirements. Input types are grouped into two categories: normal data types and complex data types, as illustrated in Table 2.

Normal data types include primitive types such as numeric types (e.g., `int`, `float`), character types (e.g., `char`, `string`), and boolean types (e.g., `bool`). These can be directly assigned and written to files during generation.

Complex data types are composed, extended, or structured forms of normal types. They are used to represent more intricate data structures and logical relationships. This category includes special file formats such as JSON, XML, and JPEG. Unlike normal types, complex data types cannot be directly written to files. Instead, they require external libraries and specialized functions to construct valid inputs.

(2) Seed Corpus Synthesis. To generate high-quality initial seeds, LASGen constructs tailored prompts for the LLM by integrating forward-sliced code with contextual knowledge about the target function. Rather than asking the LLM to output seed data directly, our approach guides it to generate Python code that writes valid input values to seed files. This strategy supports both standard and complex data types by leveraging Python's expressive syntax and its extensive third-party library ecosystem.

For normal data types, the LLM is guided to generate input values and write them directly to files. The prompt is enriched with

harness code, the target function's definition, and its forward-sliced code. The desired output format is explicitly defined as Python code that generates inputs and saves them to a specified file path. To improve the quality of reasoning, the prompt includes a multi-step CoT instruction:

① Identify the structure of the arguments from the harness.
② Analyze input constraints based on the target function and its sliced context.
③ Determine input values that are likely to trigger deeper execution paths or satisfy edge conditions.
④ Assign these values appropriately in Python code.
⑤ Implement file I/O operations to write the values to a designated seed file.

With this prompt design, LASGen successfully generates valid synergistic seeds for all harnesses with normal-type parameters.

For complex data types, the process follows a similar structure but requires external libraries. Since these inputs cannot be directly represented as literals, the LLM is instructed to import appropriate libraries and construct inputs using library-specific function calls. For instance, the `json.dump` function may be used to create JSON files, or image-related libraries may invoke the `draw` method to synthesize image files. This approach enables the generation of valid seed inputs for harnesses requiring complex data types. The prompt template used for generating Python code that synthesizes synergistic seeds is illustrated in Figure 6 in Appendix A.3.

(3) Seed Validation and Corpus Finalization. After the previous generation steps, a seed corpus is constructed for each harness. Since the fuzzing engine used by LASGen requires at least one valid seed to initiate fuzzing, it is essential to ensure the availability of executable seeds. To address cases where, due to the inherent randomness of LLM-based generation, all seeds in a corpus may be invalid, LASGen incorporates a seed filtering and evaluation module.

In this module, LASGen first checks whether a seed corpus exists for each harness. Each seed is then used to execute the corresponding harness, and the return status and output are collected. If the program terminates normally, the seed is marked as valid. Otherwise, it is considered invalid. If all seeds associated with a harness are invalid, the system triggers a corpus regeneration process. This regeneration–evaluation loop continues until a predefined retry limit is reached. If the maximum number of attempts is reached without producing a valid seed, the harness is discarded, and the system returns to the harness generation stage to produce a new candidate. Through this validation mechanism, LASGen ensures that each harness is ultimately paired with a high-quality, executable seed corpus (a set of initial seeds), enabling reliable fuzzing.

4 Implementation

We implemented LASGen in approximately 2,700 lines of Python code, encompassing both static analysis and LLM-based generation modules.

Static Analysis Module. To extract function information from the target library, we used `ctags` [13] to generate a function index, which was parsed to retrieve function signatures, source file paths, and related metadata. We then employed `cscope` [30] to build a call graph database, allowing fast lookup of function symbols and their relationships. For harness compilation, we used `Bear` [31] to capture compilation commands, which are essential for replicating the build environment. Code extraction and slicing were performed using the Clang/LLVM toolchain.

LLM-based Generation Module. For RAG, we utilized the OpenAI Text Embedding model [21] to convert text segments into high-dimensional vectors. These vectors were stored in a `Chroma` [32] vector database, whose retriever component was used to fetch relevant context during prompt construction. We adopted the `DeepSeek-V3` [6] model for code generation.

Fuzzing Configuration. We integrated `AFL++` [9], a state-of-the-art coverage-guided fuzzer, as our fuzzing engine. All harnesses were compiled using `Clang` with `AFL++` instrumentation enabled, supporting both `AddressSanitizer` [25] and `UndefinedBehavior-Sanitizer` [26]. After fuzzing, LASGen collected the results. If a crash was detected, the corresponding input and output directories were preserved. We then used `GDB` to analyze the crash and identify vulnerabilities.

5 Evaluation

We evaluated LASGen on real-world open-source libraries and answered the following research questions:

RQ1: *How effective is* LASGen *in improving fuzzing coverage on real-world libraries?*
RQ2: *To what extent does* LASGen *facilitate the generation of effective fuzzing inputs?*
RQ3: *Can* LASGen *successfully discover previously unknown vulnerabilities in real-world libraries?*

5.1 Experiment Setup

Environment. We conducted our experiments on a server equipped with a 96-core AMD EPYC 9654 processor, 1 TB of RAM, and running Ubuntu 22.04.5 LTS (x86_64).

Dataset. To evaluate the effectiveness of LASGen in library fuzzing, we constructed a benchmark dataset named *LibFuzz-Bench* (as described in the left column of Table 3). This dataset is based on the benchmark used to evaluate AFGen [18] and has been refined by increasing its scale, removing mislabeled entries, and providing accurate vulnerability annotations. *LibFuzz-Bench* contains 127 known vulnerabilities across 11 libraries, including 102 inherited from the original dataset and 25 additional vulnerabilities identified by AFGen. All listed vulnerabilities are confirmed with assigned CVE IDs.

LLM Configuration. The temperature parameter is a critical setting in LLMs, influencing the diversity and randomness of generated content by adjusting the output probability distribution [44]. Before our experiments, we performed a preliminary evaluation of different temperature values for each generation stage. A temperature of 0.6 consistently yielded more stable, context-appropriate outputs and was therefore adopted throughout the system.

Table 3: Vulnerability coverage of libraries achieved by LASGen and other fuzzers.

LibFuzz-Bench		LASGen				AFGen			Hopper				OGHarn				PromptFuzz			
Name	**CVEs**	**TP**	**FP**	**Cov**	**FPR**	**TP**	**FP**	**Cov**	**TP**	**FP**	**Cov**	**FPR**	**TP**	**FP**	**Cov**	**FPR**	**TP**	**FP**	**Cov**	**FPR**
libtiff	3	2	0	66.67%	0%	3	N/A	100%	2	3	66.67%	60.00%	0	3	0%	100%	2	6	66.67%	75.00%
ngiflib	13	13	1	100%	7.14%	10	N/A	76.92%	4	5	30.77%	55.56%	2	5	15.38%	71.43%	0	0	0%	N/A
libwav	8	5	1	62.50%	16.67%	2	N/A	25.00%	3	4	37.50%	57.14%	1	0	12.50%	0%	1	1	12.50%	50.00%
ffjpeg	13	11	0	84.62%	0%	10	N/A	76.92%	4	3	30.77%	42.86%	6	4	46.15%	40.00%	0	0	0%	N/A
tcpreplay	26	13	3	50.00%	18.75%	20	N/A	76.92%	0	0	0%	N/A	0	3	0%	100%	0	0	0%	N/A
jasper	8	7	2	87.50%	22.22%	5	N/A	62.50%	1	3	12.50%	75.00%	0	0	0%	N/A	0	0	0%	N/A
jhead	10	9	1	90.00%	10.00%	7	N/A	70.00%	6	5	60.00%	45.45%	8	4	80.00%	33.33%	7	11	70.00%	61.11%
oniguruma	1	1	0	100%	0%	1	N/A	100%	0	0	0%	N/A	0	0	0%	N/A	0	0	0%	N/A
lupng	3	3	0	100%	0%	3	N/A	100%	1	0	33.33%	0%	0	0	0%	N/A	2	3	66.67%	60.00%
libming	29	23	4	79.31%	14.81%	23	N/A	79.31%	0	0	0%	N/A	0	3	0%	100%	0	0	0%	N/A
liblouis	13	11	1	84.62%	8.33%	6	N/A	46.15%	0	0	0%	N/A	7	4	53.85%	36.36%	4	10	30.77%	71.43%
Total	127	98	13	77.17%	11.71%	90	N/A	70.87%	21	23	16.54%	52.27%	24	26	18.90%	52.00%	16	31	12.60%	65.96%

TP = True Positives; **FP** = False Positives; **Cov** = Coverage; **FPR** = False Positive Rate

Fuzzing Crash Triage. Due to the randomness in the synthesis of initial inputs by library fuzzing tools, the generated harnesses may include vulnerable functions or codes. As a result, subsequent fuzzing may fail to uncover real vulnerabilities within the target library, instead triggering crashes in the generated harness. We consider a crash a false positive (FP) if it occurs outside the target library's scope during fuzzing. We leverage Sanitizer reports and GDB to manually analyze the backtrace and memory states, tracing the root cause of each crash to determine false positives.

5.2 RQ1: Fuzzing Coverage Evaluation

To evaluate the effectiveness of LASGen in improving fuzzing coverage for libraries, we conducted vulnerability and edge coverage evaluations.

5.2.1 Vulnerability Coverage Evaluation. We compared LASGen with existing fuzzers by checking their ability to trigger known vulnerabilities in a benchmark dataset.

Setup. We used *LibFuzz-Bench* as the benchmark dataset and compared LASGen against four fuzzers, grouped into two categories: traditional fuzzers using conventional input generation methods (AFGen [18], Hopper [5], and OGHarn [28]), and LLM-based fuzzers (PromptFuzz [20]). As AFGen has not been publicly released, we adopted its results from the original paper. Since OGHarn only supports harness generation, we used LASGen to generate initial seeds and manually fed them into the fuzzing engine. All experiments were conducted in a unified environment. To reduce randomness, each vulnerable library was fuzzed for 24 hours on a dedicated CPU core, repeated five times, after which the results were averaged. Crashes were manually analyzed to confirm if they triggered a known vulnerability. We also recorded de-duplicated false positives, defined as crashes unrelated to the target function that result from ineffective input generation.

Time Performance and Query Cost. We measured the time and LLM query cost required by LASGen to generate initial inputs. On average, it took 21.9 minutes per library and 118 seconds per function. This includes 49 seconds for static analysis, 20 seconds for harness generation, and 49 seconds for initial seed generation. Regarding the query cost, the average per-library cost was $3.74.

Results. Table 3 summarizes the number of vulnerabilities discovered by each fuzzer and their coverage rates. LASGen achieved the highest coverage across 9 of 11 libraries, with an overall vulnerability coverage rate of 77.17%. For the remaining two libraries (libtiff and tcpreplay), LASGen ranked second, slightly behind AFGen, which had the next-best overall performance. In contrast, the other fuzzers showed significantly lower effectiveness across the benchmark.

There are two main reasons for this result. First, traditional fuzzers primarily target API functions and often fail to generate valid inputs for internal functions, making it difficult to uncover deep, function-level vulnerabilities through API entry points alone. Second, many fuzzers rely on complex configuration files to enable accurate analysis. In cases where such configurations were missing (e.g., tcpreplay, oniguruma), input generation was severely affected, leading to poor fuzzing outcomes. In contrast, LASGen does not require manual configuration. It automatically extracts static information from the library and utilizes it to generate high-quality initial inputs for all functions. This not only improves the tool's usability but also significantly enhances the discovery of deep-function vulnerabilities throughout the codebase.

Additionally, LASGen achieved the lowest false-positive rate among all evaluated fuzzers. Notably, a higher false-positive rate was observed in the tcpreplay and jasper libraries. We attribute this phenomenon to their complex function invocation structures, which encompass numerous crash-prone code paths and function calls. Appendix A.2 provides an illustrative example of such false positives. Although these crashes are easily triggered, they occur outside the target function and are therefore classified as false positives. These results are consistent with expectations. Since AFGen did not report false-positive data, its result is marked as `N/A`. All other fuzzers generated more false positives than LASGen.

Table 4: Edge Coverage of whole library fuzzing.

Library	Edge Coverage Comparison		
	LASGen	Hopper	PromptFuzz
libtiff	19,744 (54.40%)	16,837 (46.39%)	18,207 (50.17%)
ngiflib	919 (51.03%)	389 (21.60%)	0 (0%)
libwav	256 (60.81%)	155 (36.82%)	183 (43.45%)
ffjpeg	1,456 (76.15%)	451 (23.59%)	0 (0%)
tcpreplay	6,151 (21.90%)	0 (0%)	0 (0%)
jasper	12,240 (40.31%)	1,099 (3.62%)	167 (0.55%)
jhead	2,752 (52.07%)	1,264 (23.92%)	1,850 (35.01%)
oniguruma	16,415 (71.95%)	1,035 (4.54%)	0 (0%)
lupng	4,559 (50.92%)	911 (10.17%)	2,878 (32.14%)
libming	18,326 (60.72%)	13,481 (44.67%)	0 (0%)
liblouis	11,704 (70.27%)	9,630 (57.82%)	10,255 (61.57%)
Total	94,522 (55.61%)	45,252 (22.67%)	33,552 (17.27%)

5.2.2 Edge Coverage Evaluation. In the edge coverage evaluation, we compare the edge coverage of LASGen with fuzzers in whole-library fuzzing.

Setup. We conducted whole library fuzzing experiments on the 11 libraries in *LibFuzz-Bench* and recorded the edge coverage for each fuzzer. Due to the unavailability of AFGen and the lack of full fuzzing automation in OGHarn, only Hopper and PromptFuzz were included in the comparison. Each fuzzing session lasted 24 hours per library.

Results. Since LASGen uses multiple distinct harnesses as entry points for fuzzing, its results are stored across separate output directories. To evaluate the overall coverage, we computed the union of their fuzz bitmaps to calculate the total number of edges covered by LASGen. Further details on the edge coverage calculation are provided in Appendix A.1. Table 4 presents the number of covered edges and the edge coverage for each fuzzer. Across the 11 libraries, LASGen achieved the highest number of covered edges (94,522) and the highest edge coverage of 55.61%, which is 2.45 times that of Hopper and 3.22 times that of PromptFuzz.

Answers to RQ1

LASGen achieves the highest vulnerability coverage (77.17%), the highest edge coverage (55.61%), and the lowest false-positive rate (11.71%) across the evaluated real-world libraries, demonstrating superior effectiveness compared to existing fuzzers.

5.3 RQ2: Evaluation of Harness and Seed Generation (Ablation Study)

To assess the effectiveness of LASGen in enhancing fuzzing input generation, we conducted separate experiments for harness generation and initial seed generation.

5.3.1 Harness Generation. This experiment evaluates the quality of harnesses generated by LASGen by measuring their ability to trigger valid crashes during fuzzing.

Table 5: Results of whole library fuzzing.

Library	LASGen		Hopper		PromptFuzz	
	TH	FH	TH	FH	TH	FH
libtiff	38	15	17	23	28	18
ngiflib	13	2	4	4	0	0
libwav	7	0	2	0	3	4
ffjpeg	21	3	10	15	0	0
tcpreplay	23	3	0	0	0	0
jasper	67	22	55	199	0	0
jhead	12	0	13	20	21	58
oniguruma	122	35	57	24	0	0
lupng	19	7	4	6	98	46
libming	185	39	30	44	0	0
liblouis	99	23	4	21	77	145
Total	604	149	196	356	227	271
Precision	80.21%		35.51%		45.58%	

TH is the number of harnesses that trigger valid crashes.
FH is the number of harnesses that trigger false-positive crashes.

Setup. We conducted fuzzing experiments on all functions from the 11 libraries in *LibFuzz-Bench*. We selected Hopper and PromptFuzz as baselines and assessed harness quality by checking the validity of the triggered crashes. Each fuzzing session lasted 24 hours per library, and all crashes were manually verified.

Results. Table 5 reports the number of harnesses that triggered valid crashes (`TH`) and false-positive harnesses (`FH`) for each tool. LASGen achieved the highest precision, generating 604 valid harnesses that led to 775 unique crashes, with 80.2% precision. This is 2.26× higher than Hopper and 1.76× higher than PromptFuzz. Some libraries (e.g., ngiflib, tcpreplay) had zero `TH` and `FH` values for the baseline fuzzers due to missing configuration, but this did not affect the overall precision comparison.

5.3.2 Seed Generation. To assess the effectiveness of the synergistic seeds generated by LASGen, we compared the time required to trigger target crashes using LASGen-generated seeds versus those generated by traditional methods.

Setup. We randomly selected four vulnerable functions from each library in *LibFuzz-Bench*, totaling 44 functions. For each, we generated valid harnesses from LASGen. Then we created initial seeds using four approaches: LASGen, the LLM-based seed generation method guided by input formats G^{2}Fuzz [43], the symbolic execution framework Angr [29], and a basic LLM prompt. The Angr script used the target function's call as its path constraint. The basic LLM prompt used `DeepSeek-V3` model and comprised simple instructions that provided the harness and asked the LLM to generate initial seeds. The harnesses and corresponding seeds were then fuzzed using AFL++. Each fuzzing session lasted 8 hours, and we recorded the time to discover the target crash. This process was repeated five times for each seed, and the reported time is the average of these runs. If a tool failed to generate a seed or did not trigger a crash within the time limit, the result was marked as a timeout (`T.O.`).

Table 6: Time consumption for crash detection in different seed generation.

Library	LASGen (H^-S^+)				G^2Fuzz				Angr				DeepSeek-V3			
	F1	F2	F3	F4	F1	F2	F3	F4	F1	F2	F3	F4	F1	F2	F3	F4
libtiff	0.01s	12913.54s	0.01s	0.05s	T.O.	27107.43s	T.O.	1631.77s	*T.O.	T.O.	*T.O.	3.00s	T.O.	T.O.	T.O.	336.92s
ngiflib	0.01s	7.89s	0.02s	0.55s	13.65s	36.32s	3.54s	80.15s	3.31s	124.16s	0.17s	*T.O.	4.98s	208.42s	109.45s	1.49s
libwav	3.18s	75.38s	11.54s	15.66s	36.77s	832.43s	6649.07s	474.40s	3.32s	1202.67s	44.42s	T.O.	T.O.	T.O.	51.08s	T.O.
ffjpeg	2.42s	0.04s	0.57s	0.06s	12699.67s	0.38s	0.97s	0.06s	2.43s	*T.O.	1.23s	0.43s	T.O.	1.87s	3.40s	58.49s
tcpre.	1.54s	6.02s	0.04s	0.40s	4.65s	12.22s	0.07s	3.98s	*T.O.	*T.O.	0.06s	35.55s	1.92s	5.55s	1701.67s	5.23s
jasper	0.01s	0.06s	0.03s	0.20s	10.65s	1.28s	0.14s	4.15s	*T.O.	*T.O.	*T.O.	T.O.	13.00s	3.29s	0.55s	0.58s
jhead	0.88s	13559.65s	1.42s	600.56s	T.O.	T.O.	41.28s	602.44s	*T.O.	*T.O.	*T.O.	1083.89s	0.94s	26245.20s	321.37s	1446.69s
onig.	0.01s	0.03s	0.01s	0.77s	0.04s	0.17s	T.O.	4.66s	0.02s	*T.O.	T.O.	36.33s	0.66s	0.52s	T.O.	43.07s
lupng	246.04s	0.03s	8.53s	0.82s	1922.47s	1.05s	16.98s	39.65s	254.29s	0.15s	16.76s	T.O.	T.O.	1.06s	33.19s	192.24s
libming	379.62s	0.03s	2.53s	0.08s	6403.41s	5.81s	5.94s	0.15s	*T.O.	1.00s	4.53s	26.60s	382.28s	2.06s	337.64s	1.57s
liblouis	9.42s	600.10s	600.12s	600.60s	51.57s	606.38s	T.O.	601.62s	9.51s	*T.O.	*T.O.	1917.14s	600.19s	641.32s	774.36s	T.O.
AVG	35.70s				501.56s				77.28s				274.77s			

H^-S^+ denotes 'seed-only ablation given a fixed harness set' (harnesses are generated once and reused).
F1 - F4 are randomly selected library functions for evaluation.

Results. Table 6 shows the time each method took to trigger crashes. Entries marked with * indicate seed generation failures. LASGen successfully generated seeds for all 44 harnesses and triggered every target crash within the time limit. In contrast, Angr detected only 23 crashes, failing to generate seeds in 16 cases and timing out in 5, mainly due to path explosion and constraint-solving complexity. G^2Fuzz and the basic LLM prompt method successfully generated seeds for all harnesses but triggered only 38 and 34 crashes, respectively. This is because both methods lacked guidance on path constraints when using LLMs for seed generation, leading to poor performance in triggering crashes along complex execution paths.

We also compared the average time required by each method to discover crashes. To ensure fairness, the comparison only included cases where all four methods succeeded (highlighted in pink in Table 6). LASGen achieved the shortest average crash discovery time of 35.70 seconds, outperforming G^2Fuzz by 92.88%, Angr by 53.80%, and the basic LLM prompt method by 87.01%.

Answers to RQ2

LASGen achieved the highest harness generation precision (80.21%) and successfully generated valid seeds for all target functions, enabling it to discover every crash. It also reduced crash discovery time by over 50% compared to G^2Fuzz, Angr, and the basic LLM method, confirming the advantage of LASGen in generating high-quality fuzzing inputs.

5.4 RQ3: Real-World Vulnerability Discovery

To assess LASGen's ability to discover previously unknown vulnerabilities in real-world libraries, we conducted comprehensive fuzzing and manually analyzed the resulting crashes.

Setup. The evaluation included 17 libraries: the latest versions of the 11 libraries from *LibFuzz-Bench* and 6 additional libraries (LIEF, feh, libheif, libjxl, jsonxx, and fast-cpp-csv-parser). Each library was fuzzed for 24 hours. All crashes were examined to confirm that the vulnerabilities were exploitable and previously undisclosed.

```
01 SDL_Surface * SDL_LoadGIF(const char * file) {
02   ...
03  surface = SDL_CreateRGBSurface(SDL_SWSURFACE |
        SDL_SRCCOLORKEY, gif->width, gif->height, 8, 0,0,0,0);
04  if (surface == NULL) {
05      ...
06      return NULL; }
07  psrc = p; pdst = surface->pixels;
08  for(i=0; i<gif->height; i++)
09  {
10      ngiflib_memcpy(pdst, psrc, gif->width);
```

(a) A null pointer dereference bug in ngiflib

```
01 int TIFFYCbCrToRGBInit(TIFFYCbCrToRGB *ycbcr, float *luma,
   float *refBlackWhite) {
02     TIFFRGBValue *clamptab;
03     ...
04     clamptab = (TIFFRGBValue *)((uint8_t *)ycbcr +
         TIFFroundup_32(sizeof(TIFFYCbCrToRGB), sizeof(long)));
05     _TIFFmemset(clamptab, 0, 256);
```

(b) A buffer overflow bug in libtiff

Figure 4: Vulnerable cases discovered by LASGen.

Results. LASGen uncovered 33 previously unknown vulnerabilities across 17 libraries, including buffer overflows, integer overflows, and null pointer dereferences. Following responsible disclosure practices, we reported all vulnerabilities with proof-of-concept inputs to the project maintainers. At the time of writing, 26 of these vulnerabilities have been confirmed and patched by the respective maintainers, with 8 assigned CNVD IDs. The remaining seven are awaiting confirmation. A summary is provided in Table 7, omitting details for unpatched issues.

Case Study 1. Figure 4(a) shows a null pointer dereference in a GIF parser. The vulnerability stems from the lack of validation of key parameters (`width`, `height`), resulting in a `surface` object

Table 7: LASGen**'s newly-found real-world vulnerabilities.**

ID	Library	Vulnerable Function	Location	Bug Type	Status
1.	libtiff	_TIFFmemcpy	libtiff/tif_unix.c:357	Heap Buffer Overflow	Confirmed
2.	libtiff	_TIFFmemset	libtiff/tif_color.c:263	Stack Buffer Overflow	Confirmed
3.	libtiff	printTIF	tools/fax2ps.c:255	Integer Overflow	Confirmed
4.	ngiflib	SDL_LoadGIF	ngiflibSDL.c:99	Null Pointer Dereference	Confirmed
5.	ngiflib	SDL_LoadGIF	ngiflibSDL.c:92	Null Pointer Dereference	Confirmed
6.	ngiflib	CheckGif	ngiflib.c:409	Buffer Overflow	Confirmed
7.	ngiflib	manage_event	SDLLaffgif.c:15	Buffer Overflow	Confirmed
8.	ngiflib	SDL_LoadAnimatedGif	ngiflibSDL.c:220	Buffer Overflow	Confirmed
9.	ngiflib	GifImgDestroy	ngiflib.c:28	Buffer Overflow	Confirmed
10.	oniguruma	onigenc_mb2_is_code_ctype	src/regenc.c:925	Improper Input Validation	Confirmed
11.	oniguruma	onigenc_mb4_is_code_ctype	src/regenc.c:940	Improper Input Validation	Confirmed
12.	oniguruma	is_code_ctype	src/iso8859_1.c:249	Improper Input Validation	Confirmed
13.	oniguruma	koi8_r_is_code_ctype	src/koi8_r.c:125	Improper Input Validation	Confirmed
14.	oniguruma	koi8_is_code_ctype	src/koi8.c:126	Improper Input Validation	Confirmed
15.	liblouis	_lou_pattern_check	liblouis/pattern.c:1616	Stack Overflow	Confirmed
16.	liblouis	checkEmphasisChange	liblouis/lou_translateString.c:1838	Heap Buffer Overflow	Confirmed
17.	liblouis	copyCharacters	liblouis/lou_translateString.c:924	Stack Buffer Overflow	Confirmed
18.	liblouis	releaseStringBuffer	liblouis/lou_translateString.c:96	Null Pointer Dereference	Confirmed
19.	LIEF	header	include/LIEF/Abstract/Binary.hpp:115	Buffer Overflow	Confirmed
20.	feh	init_list_mode	src/list.c:59	Double Free	Confirmed
21.	feh	feh_load_image	src/imlib.c:352	Integer Overflow	Confirmed
22.	libheif	heif_image_set_raw_color_profile	libheif/api/libheif/heif.cc:2156	Buffer Overflow	Confirmed
23.	libjxl	DecodeImageEXR	lib/extras/dec/exr.cc:159	Integer Overflow	Confirmed
24.	jsonxx	parse_number	jsonxx.cc:201	Integer Overflow	Confirmed
25.	fast-cpp-csv-parser	read_row	csv.h:1168	Buffer Overflow	Confirmed
26.	fast-cpp-csv-parser	read_header	csv.h:1089	Buffer Overflow	Confirmed

with a `NULL` `pixels` field. Although a check exists on Line 4, it is bypassed, and a subsequent memory-copy operation on Line 10 dereferences a null pointer, causing a crash. An attacker could exploit this vulnerability by providing a maliciously crafted GIF file to cause denial-of-service (DoS).

Case Study 2. Figure 4(b) presents a buffer overflow bug. The vulnerable function fails to validate the size of the `ycbcr` structure. If the input exceeds the expected allocation, the `clamptab` pointer may reference out-of-bounds memory. When `_TIFFmemset` writes to this invalid location, it causes a stack-based overflow. This could be exploited to cause DoS or to hijack the control flow and execute arbitrary code on the target system.

Answers to RQ3

LASGen uncovered 33 previously unknown vulnerabilities across 17 real-world libraries. Among them, 26 have been confirmed and patched by developers, with eight assigned CNVD IDs. These findings demonstrate the effectiveness of LASGen in discovering real-world zero-day vulnerabilities.

6 Discussion

The evaluation of LASGen demonstrates the effectiveness in high-coverage library fuzzing. To give insights into more effective fuzzing approaches, we outline potential threats to the validity of our research findings and suggest directions for future work.

6.1 Limitations of Code Slicing

Sliced code plays a crucial role in LASGen 's input generation process. With the help of the `Clang/LLVM` toolchain, we generate relatively complete slices for functions in the target library. However, some code constructs may reduce slicing accuracy. For example, preprocessor directives (e.g., `#define`, `#ifdef`) are resolved before compilation, and their semantics are not retained in LLVM IR. In these special cases, our slicer may miss parts of the function's logic, leading to incomplete slices and, consequently, affecting the LLM's generation. Custom slicing rules are needed to handle these cases more accurately. In addition, forward slicing includes the entire call chain of a function. For a small number of functions with complex call patterns, this can produce large slices that exceed LLM prompt limits, affecting reasoning quality. While we apply pruning strategies to reduce prompt length, this may also remove important context. More precise filtering techniques will be explored in future work.

6.2 Instability of LLM-Generated Results

LASGen integrates static analysis with LLMs to generate initial inputs. While LLMs can improve the efficiency of reasoning for input generation, they also introduce unique risks and limitations compared to traditional program analysis tools. LLMs are inherently probabilistic and lack correctness guarantees. Hallucinations [40] frequently lead to syntax or logic errors, resulting in compilation failures or false positives. In contrast, program analysis techniques such as symbolic execution employ SMT solvers to directly enumerate program paths and solve constraints. Although these tools may face path explosion challenges, they provide deterministic results, making them less error-prone than LLMs. In the future, we may explore combining symbolic execution with LLMs or introducing a multi-model voting mechanism to mitigate such problems. Furthermore, querying LLMs during large-scale fuzzing incurs substantial computational overhead, leading to greater resource consumption than traditional tools. Meanwhile, we also find that generation quality also depends on the chosen LLM [37]. More advanced models tend to produce better results, but selecting the most suitable model for different tasks remains an open research problem.

6.3 Language Support Limitations

LASGen relies on the `Clang/LLVM` toolchain for static analysis, which primarily supports `C/C++` libraries. Supporting other languages would require building dedicated static analysis modules. As an alternative, LLMs may be able to perform slicing directly on source code [27], eliminating the need for language-specific tooling and enabling broader language compatibility. We leave this direction for future exploration.

7 Related Work

Here, we review prior work on fuzzing improvements, focusing on harness generation and seed generation.

7.1 Fuzzing Harness Generation

Automatic harness generation for libraries has long been a key topic in fuzzing research, with various methods proposed to improve harness quality. FUDGE [3] slices client code to construct harness contexts. FuzzGen [15] analyzes and merges API dependencies to generate harnesses with various API interactions. GraphFuzz [12] builds dataflow graphs from manually written harnesses and performs graph-based mutations to create new harnesses. Hopper [5] learns API constraints and applies coverage-guided, syntax-aware mutations to evolve inputs. OGHarn [28] extracts rich function metadata to generate API call sequences. However, these tools primarily invoke library APIs, thereby limiting coverage of internal functions. To address this, AFGen [18] introduces whole-function fuzzing by slicing all functions and synthesizing harnesses. Yet, it relies on iterative compilation-error-driven inference of variable constraints, affecting efficiency and correctness.

Recently, LLMs have been explored to improve the harness generation process. OSS-Fuzz-Gen [17] integrates with the Google OSS-Fuzz [24] infrastructure, leveraging LLMs together with contextual information from OSS-Fuzz projects to construct fuzzing targets. However, it can only operate on projects already integrated into OSS-Fuzz, limiting its applicability to other libraries. LLM4FDG [42] enhances the quality of harnesses by designing prompts and configuring contexts. Fuzz4All [36] refines prompts and generates harnesses by extracting information from resources and using iterative feedback. PromptFuzz [20] uses coverage feedback to guide prompt updates and function combinations. CKGFuzzer [38] builds a function knowledge graph for LLM-based harness synthesis. However, these methods face challenges because the context fed to LLMs is often imprecise and redundant, leading to hallucinations and compromising the correctness of the generated harnesses. In addition, most approaches prioritize composition over whole-function coverage, thereby limiting overall code coverage.

7.2 Fuzzing Initial Seed Generation

Previous work has improved initial seed generation by techniques such as taint analysis to trace data flow and guide seed mutation, as in GREYONE [10], or symbolic execution to solve path constraints [34]. Yet, these approaches often suffer from path explosion and inefficiency, which significantly impact the effectiveness of initial seed generation. Skyfire [33] uses a data-driven approach to generate structured inputs by learning probabilistic context from samples. SLF [41] and Learn&Fuzz [11] employ a valid field mutation strategy and a machine learning-based approach, respectively, to generate seeds from a dataset. However, they will face significant challenges in library fuzzing scenarios, as obtaining an input sample set for library functions is not straightforward. The work [16] highlights the potential of LLMs for generating seeds and explores a hybrid approach that combines LLMs with constraint-based tools to improve generation quality. G^2Fuzz [43] leverages LLMs to synthesize input generators that produce seeds conforming to the expected input format and relies on fuzzers for further mutation. However, these methods overlook the role of prompt context in seed generation and lack the contextual guidance needed to steer input generation toward deeper program states or execution paths. This limitation may hinder their effectiveness in uncovering vulnerabilities hidden behind complex semantic constraints, and the LLM's capability for data-flow analysis of inputs was not fully utilized.

8 Conclusion

In this paper, we presented LASGen, an automated library fuzzing framework that combines static analysis and LLMs to support whole-function fuzzing via synergistic harness–seed co-synthesis. By combining library-specific static knowledge with LLM-driven input generation and self-repair, LASGen can produce high-quality harnesses and synergistic seeds without manual effort. Evaluated on a benchmark of 127 vulnerabilities across 11 libraries, LASGen outperformed existing fuzzers in both coverage and input quality. When LASGen was applied to 17 real-world libraries, it discovered 33 previously unknown vulnerabilities, 26 of which have been confirmed and patched. These results demonstrate the effectiveness of LASGen in advancing automated, LLM-assisted library fuzzing.

Acknowledgements

We thank the insightful review comments. This work was supported by the Natural Science Foundation of Shandong Province (Grant No. ZR2025MS991). Jiongyi Chen was supported by the National Natural Science Foundation of China (Grant No. 62302508).

References

[1] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. NAUTILUS: Fishing for Deep Bugs with Grammars. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS), San Diego, California, USA, February 24-27, 2019.*

[2] Orlando Ayala and Patrice Béchard. 2024. Reducing hallucination in structured outputs via Retrieval-Augmented Generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Track (NAACL), Mexico City, Mexico, June 16-21, 2024.*

[3] Domagoj Babic, Stefan Bucur, Yaohui Chen, Franjo Ivancic, Tim King, Markus Kusano, Caroline Lemieux, László Szekeres, and Wei Wang. 2019. FUDGE: fuzz driver generation at scale. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT) (FSE), Tallinn, Estonia, August 26-30, 2019.*

[4] Anh Thu Maria Bui, Saskia Felizitas Brech, Natalie Hußfeldt, Tobias Jennert, Melanie Ullrich, Timo Breuer, Narjes Nikzad-Khasmakhi, and Philipp Schaer. 2024. The Two Sides of the Coin: Hallucination Generation and Detection with LLMs as Evaluators for LLMs. In *Proceedings of the Working Notes of the Conference and Labs of the Evaluation Forum (CLEF), Grenoble, France, 9-12 September, 2024.*

[5] Peng Chen, Yuxuan Xie, Yunlong Lyu, Yuxiao Wang, and Hao Chen. 2023. Hopper: Interpretative Fuzzing for Libraries. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), Copenhagen, Denmark, November 26-30, 2023.*

[6] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, and etc. 2024. DeepSeek-V3 Technical Report. *CoRR* (2024).

[7] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Seattle, WA, USA, July 17-21, 2023.*

[8] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large Language Models are Edge-Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), Lisbon, Portugal, April 14-20, 2024.*

[9] Andrea Fioraldi, Dominik Christian Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT), August 11, 2020.*

[10] Shuitao Gan, Chao Zhang, Peng Chen, Bodong Zhao, Xiaojun Qin, Dong Wu, and Zuoning Chen. 2020. GREYONE: Data Flow Sensitive Fuzzing. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security), August 12-14, 2020.*

[11] Patrice Godefroid, Hila Peleg, and Rishabh Singh. 2017. Learn&Fuzz: machine learning for input fuzzing. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), Urbana, IL, USA, October 30 - November 03, 2017.*

[12] Harrison Green and Thanassis Avgerinos. 2022. GraphFuzz: Library API Fuzzing with Lifetime-aware Dataflow Graphs. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, May 25-27, 2022.*

[13] Darren Hiebert. 2015. *Ctags: ctags generates an index (or tag) file of language objects found in source files for programming languages.* Retrieved October 28, 2025 from `https://ctags.io`

[14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* (2025).

[15] Kyriakos K. Ispoglou, Daniel Austin, Vishwath Mohan, and Mathias Payer. 2020. FuzzGen: Automatic Fuzzer Generation. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security), August 12-14, 2020.*

[16] Zongze Jiang, Ming Wen, Jialun Cao, Xuanhua Shi, and Hai Jin. 2024. Towards Understanding the Effectiveness of Large Language Models on Directed Test Input Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), Sacramento, CA, USA, October 27 - November 1, 2024.*

[17] Dongge Liu, Oliver Chang, Jonathan Metzman, Martin Sablotny, and Mihai Maruseac. 2024. *OSS-Fuzz-Gen: Automated Fuzz Target Generation.* `https://github.com/google/oss-fuzz-gen`

[18] Yuwei Liu, Yanhao Wang, Xiangkun Jia, Zheng Zhang, and Purui Su. 2024. AFGen: Whole-Function Fuzzing for Applications and Libraries. In *Proceedings of the IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, May 19-23, 2024.*

[19] Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. 2021. Boosting symbolic execution via constraint solving time prediction (experience paper). In *Proceedings of the ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Virtual Event, Denmark, July 11-17, 2021.*

[20] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt Fuzzing for Fuzz Driver Generation. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS), Salt Lake City, UT, USA, October 14-18, 2024.*

[21] OpenAI. 2022. *Introducing Text and Code Embeddings.* Retrieved October 1, 2025 from `https://openai.com/index/introducing-text-and-code-embeddings`

[22] Nam H. Pham, Tung Thanh Nguyen, Hoan Anh Nguyen, and Tien N. Nguyen. 2010. Detection of Recurring Software Vulnerabilities. In *Proceedings of the ASE 2010, 25th IEEE/ACM International Conference on Automated Software Engineering (ASE), Antwerp, Belgium, September 20-24, 2010.*

[23] Kostya Serebryany. 2015. *LLVM libFuzzer.* Retrieved October 30, 2025 from `https://llvm.org/docs/LibFuzzer.html`

[24] Kostya Serebryany. 2017. OSS-Fuzz - Google's continuous fuzzing service for open source software. USENIX Association, Vancouver, BC.

[25] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC), Boston, MA, USA, June 13-15, 2012.*

[26] Kostya Serebryany and LLVM Contributors. 2012. *Undefined Behavior Sanitizer.* Retrieved October 1, 2025 from `https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html`

[27] Kimya Khakzad Shahandashti, Mohammad Mahdi Mohajer, Alvine Boaye Belle, Song Wang, and Hadi Hemmati Lassonde. 2025. Program Slicing in the Era of Large Language Models. In *Proceedings of the 49th IEEE Annual Computers, Software, and Applications Conference (COMPSAC), Toronto, ON, Canada, July 8-11, 2025.*

[28] Gabriel Sherman and Stefan Nagy. 2025. No Harness, No Problem: Oracle-guided Harnessing for Auto-generating C API Fuzzing Harnesses. In *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE).*

[29] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Krügel, and Giovanni Vigna. 2016. SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016.*

[30] Joe Steffen. 2000. *Cscope: cscope allows searching code for all references to a symbol, global definitions, functions called by a function and functions calling a function.* Retrieved October 4, 2025 from `https://cscope.sourceforge.net`

[31] Peter Szabo. 2013. *Bear: Bear is a tool that generates a compilation database for clang tooling.* Retrieved October 1, 2025 from `https://github.com/rizsotto/Bear`

[32] Anton Troynikov and Jeff Huber. 2023. *Chroma: The AI-native open-source embedding database.* Retrieved October 1, 2025 from `https://www.trychroma.com`

[33] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. 2017. Skyfire: Data-Driven Seed Generation for Fuzzing. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA, May 22-26, 2017.*

[34] Weiguang Wang, Hao Sun, and Qingkai Zeng. 2016. SeededFuzz: Selecting and Generating Seeds for Directed Fuzzing. In *Proceedings of the 10th International Symposium on Theoretical Aspects of Software Engineering (TASE), Shanghai, China, July 17-19, 2016.*

[35] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP), Monticello, IL, USA, 2023.*

[36] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), Lisbon, Portugal, April 14-20, 2024.*

[37] Yu Xia, Fang Kong, Tong Yu, Liya Guo, Ryan A. Rossi, Sungchul Kim, and Shuai Li. 2024. Which LLM to Play? Convergence-Aware Online Model Selection with Time-Increasing Bandits. In *Proceedings of the ACM on Web Conference 2024 (WWW), Singapore, May 13-17, 2024.*

[38] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph. In *Companion Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, April 27 - May 3, 2025.*

[39] Xu Yang, Guofeng Zhang, Ziqi Shuai, Zhenbang Chen, and Ji Wang. 2025. Symbolic execution of floating-point programs: How far are we? *J. Syst. Softw.* (2025).

[40] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Munan Ning, and Li Yuan. 2023. LLM Lies: Hallucinations are not Bugs, but Features as Adversarial Examples. *CoRR* (2023).

[41] Wei You, Xuwei Liu, Shiqing Ma, David Mitchel Perry, Xiangyu Zhang, and Bin Liang. 2019. SLF: fuzzing without valid seed inputs. In *Proceedings of the 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, May 25-31, 2019.*

[42] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. How Effective Are They? Exploring

```
#include ...

int main(int argc, char *argv[]) {
    ...
    // Open input file
    FILE *input = fopen(argv[1], "rb");
    ...
    // Create output stream
    jas_stream_t *out = jas_stream_memopen(0, 0);
    // Create test image
    jas_image_t *image = jas_image_create();
    // Create BMP info structure
    bmp_info_t *info = bmp_info_create();
    ...
    // Read test data from input file
    // Format: width(4) height(4) depth(4) numcmpts(4) cmpts[3]
    int cmpts[3] = {0};
    ...
    int numcmpts;
    if (fread(&numcmpts, sizeof(int), 1, input) != 1) {
        ...
    }
    if (fread(cmpts, sizeof(int), numcmpts, input) != numcmpts) {
        bmp_info_destroy(info);
        jas_image_destroy(image);
        jas_stream_close(out);
        fclose(input);
        return 1;
    }
    ...

    // Call the target function
    int ret = bmp_putdata(out, info, image, cmpts);
    ...
```

Listing 2: The false-positive case in the jasper library.

Large Language Model Based Fuzz Driver Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Vienna, Austria, September 16-20, 2024.*

[43] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. 2025. Low-Cost and Comprehensive Non-textual Input Fuzzing with LLM-Synthesized Input Generators. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security), Seattle, WA, USA, August 13-15, 2025.*

[44] Yuqi Zhu, Jia Li, Ge Li, Yunfei Zhao, Jia Li, Zhi Jin, and Hong Mei. 2024. Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence (EAAI), February 20-27, 2024, Vancouver, Canada.*

A Appendix

A.1 Edge Coverage Calculation

To evaluate the edge coverage of LASGen, we collected the fuzz bitmaps from the output directories generated during the fuzzing of various harnesses. By computing the union of these bitmaps, we determined the total number of edges covered by LASGen within the target library. Note that because the harnesses are also instrumented, this union inherently includes the internal edges of the harnesses. However, since a harness is a wrapper for invoking target functions, the number of edges it contains is negligible compared to the complex logic of the target library. Furthermore, the harness edges directly drive the target library functions, representing valid execution paths. Consequently, while the inclusion of harness edges introduces a marginal variance to the total edge count, it reflects the effectiveness of the fuzzing process and does not skew the evaluation of edge coverage.

A.2 False-Positive Case

Listing 2 shows a false-positive case from the jasper library, where LASGen generated a harness for `bmp_putdata` without proper input validation. The harness allocates the `cmpts` array with a fixed size of three (line 17) but reads values based on the input-controlled `numcmpts` (line 23). When `numcmpts` exceeds three, `fread` writes past the array boundary, causing a stack buffer overflow and crash.

A.3 Prompts for Generation

Figures 5 and 6 present the LLM prompt templates used for generating harnesses and initial seeds. These templates incorporate several proven prompt engineering strategies. The prompts employ an instructional and structured design, utilizing specific tags to partition the prompt into distinct modules, such as Role, Instruction, Output Format, and Example. Specifically, in the Role module, we apply the role prompting technique [35] to explicitly assign the LLM the role of a fuzzing expert. Specifying this domain-expert role guides the LLM to focus its attention mechanism and internal knowledge retrieval on the field of software security and fuzzing. This shifts the conditional probability distribution of the model's output, aligning it tightly with professional fuzzing expertise, which enhances the effectiveness of the generated fuzzing inputs.

Role:

Now you are a fuzzing expert, and you are very good at generating function harness for AFL++ (American Fuzzy Lop++).

Instruction:

The following <Sliced code> is a sliced code showing how to call the {slice_fun} function. Your task is to repair the sliced code and reconstruct it into a complete, compilable, and executable invocation of the function {slice_fun}. Finally, you will obtain the complete driver code for fuzzing the function {slice_fun}. Please refer to the following chain of thought and generation procedure:

1. Identify the variables required to invoke the function from the <Sliced code>, and initialize all necessary preconditions and auxiliary variables prior to calling {slice_fun}.

...

9. Ensure that all resources allocated during the invocation of {slice_fun} are released upon completion of the function call.

Output format:

Please implement it using {code_type} code. The final structure should follow the format illustrated below:

```
#include <>   // Library header file
int main()     // Entry point that accepts input files
{
    // Invocation of the library functions
}
```

Input:

<{slice_fun} function's definition code>: [{slice_fun_code}]
<Sliced code>: [{sliced_code}]

Example: ...

Figure 5: The prompt template for generating harness.

Role:

Now you are a fuzzing expert, and you are very good at generating initial seeds for fuzzing harnesses.

Instruction:

You are now required to generate a Python script. The purpose of the script is to construct the input file for [harness_code]. The constructed file must be written to [{init_seed_file_path}] path and will serve as the initial seed for subsequent fuzzing of [harness_code]. The reasoning process is as follows:

1. Identify the input type expected by [harness_code] and analyze its input-handling logic.
2. Integrate the information from [slice_fun_code] and [sliced_forward_code] to examine the data flow of the {slice_fun} function. Then you need to derive an initial input capable of exercising deeper execution paths within {slice_fun}.
3. Implement a Python script that, based on the input type required by [harness_code], invokes the appropriate file-generation libraries and third libraries to produce the input devised in Step 2.

...

Output format:

Output only the Python code wrapped with ```python ```.

Input:

[harness_code]: [{harness_code}]
[slice_fun_code]: [{slice_fun_code}]
[sliced_forward_code]: [{sliced_forward_code}]

Example: ...

Figure 6: The prompt template for generating Python code that synthesizes synergistic seeds.